低空经济 · 可审计数据分析 Agent

从 0 读懂这个项目

低空无人机配送 + 会自己查账的 AI 分析助手

零基础详细讲解 / 不需要任何前置知识

含生活类比、逐表拆解、四轮实验、动手步骤与术语表

配套代码与全部实测结果见 project/ 目录

2026 年 11 月

从 0 读懂这个项目：低空无人机配送 + 会自己查账的 AI 分析助手

目录

从 0 读懂这个项目：低空无人机配送 + 会自己查账的 AI 分析助手

- 0 写在最前面
- 1 先从一个奶茶店说起
- 2 认识这个开源项目
- 3 低空经济到底在讲什么
- 4 数据从哪来，哪些是真、哪些是造的
- 5 五张表长什么样
- 6 三个新工具，一个一个拆
- 7 验证器到底怎么保证数字可信
- 8 评测怎么设计才不骗人
- 9 四轮实验，一轮一轮讲
- 10 一页看懂全部数字
- 11 在你的电脑上把它跑起来
- 12 写在简历上 / 面试怎么讲
- 13 诚实清单：局限与还没做完的事
- 14 术语表
- 15 文件地图：哪个文件是干什么的

0 写在最前面

0.1 这份文档是写给谁的

写给**完全没有接触过这个项目、也不熟悉「可审计分析 Agent」「DuckDB」「验证器」这些词**的人。你只需要会一点基本的电脑操作，知道什么是表格、什么是文件夹就够了。文中第一次出现的每个术语都会先用人话解释一遍，最后还附了一张术语表。

读完之后你应该能回答：

- 这个项目到底在解决什么问题？
- 数据从哪来、哪些是真哪些是造的？
- 「模型不碰算术」是什么意思，为什么重要？
- 三个新工具分别算什么？
- 那个「验证器」凭什么说数字可信？
- 24 道题的评测是怎么设计的，为什么没有靠「让对手变笨」来刷分？
- 四轮实验分别发现了什么？
- 换一台电脑，我怎么把它跑起来、继续做下去？

0.2 一句话版本

> 我们给一个「AI 不许自己算数、每个数字都要被程序重新验一遍」的开源分析工具，装上了低空无人机配送的专属能力；然后用 24 道题、四轮实验，诚实地量出「把业务口径写进代码」到底带来了什么。

0.3 阅读地图

如果你想……	直接看
先知道这事儿为什么值得做	第 1 章
搞懂开源基座在干嘛	第 2 章
搞懂低空经济和数据	第 3、4、5 章
搞懂三个工具	第 6 章
搞懂「数字可信」的机制	第 7 章
搞懂实验怎么设计、结果是什么	第 8、9、10 章
自己动手跑	第 11 章
写简历 / 准备面试	第 12 章

---

1 先从一个奶茶店说起

1.1 一个很常见的翻车

假设你开奶茶店，问一个 AI：「我这个月比上个月少赚了多少钱？」

AI 很流畅地回答：「少了 8,500 元，主要是批发渠道下滑导致的。」

你可能就信了。但问题是：**这个 8,500 是它自己「算」出来的，还是查出来的？** 大语言模型的本质是预测下一个最像样的字，它没有真的打开你的账本做减法。它可能把上个月的数记反了、把「毛利」当「营收」、把单位看成美元，或者干脆编一个听起来合理的数。

这类错误最麻烦的地方是：**它错得很有礼貌、很自信，而且你看不出来。**

1.2 一个朴素的办法：分工

把工作拆成两半：

- **AI 负责「听懂人话」和「把结论说清楚」**：它擅长理解「少赚了多少钱」是在问差额，也擅长把结果写成一段人能读的话。
- **程序负责「算」**：数据库（本项目用 DuckDB）老老实实执行 SQL，算出 44,100 和 36,200，再做减法得到 −7,900。

- **再找一个独立的「查账员」**：报告发出去之前，把每个数字重新算一遍。凡是对不上的，整份报告不许发。

这就是这个项目最核心的一句话：**模型不碰算术**（the model never does arithmetic）。

1.3 「查账员」为什么不能是 AI 自己

如果让 AI 自己检查自己算得对不对，就像让同一个学生自己给自己批卷子。所以查账员必须是**确定性的代码**：同样的输入永远得到同样的输出，不抽样、不猜、不受心情影响。

本项目里这个查账员叫 `verify`（验证器）。它有一套 V1 到 V8 的规则，其中最重要的是 **V2：把每个数字重新算一遍。**

1.4 那这篇文档后面在讲什么

我们拿这套「不许自己算数」的工具，去做一件它原本不会的事：**分析低空无人机配送。**它原本只会算「销售额降了多少」这类通用商业问题，不会判断「一架无人机有没有飞进禁飞区」。

于是我们要解决三个新难题：

1. 怎么把业务口径（什么叫「一次冲突」、什么叫「巡航高度层」）写死在代码里，而不是每次靠人临时解释；
2. 几何计算（点在多边形里吗）不是普通加减法，怎么还能让只会 SQL 的查账员复核；
3. 怎么设计一场**不骗人**的实验，证明这些改动真的有用。

---

2 认识这个开源项目

2.1 它叫什么、从哪来

项目叫 `gfirik/agentic-analytics`，在 GitHub 上公开，Apache-2.0 协议（可以放心用于学习和二次开发）。它是 2026 年 10 月才发布的 v0.1 早期项目，作者一人维护，star 数很少——这一点我们从不掩饰，面试时也如实说。

选它的理由不是名气，而是**它的规则很干净**：所有数字都必须留下一份可复核的「计算记录」，贡献边界因此能被清楚界定（我们改了什么、没改什么，一眼可见）。

2.2 它的工作流程

原项目不是「你问一句它答一句」的聊天机器人，而是「你丢给它一堆文件，它产出一份完整报告」。流程是一条固定流水线：

```
读文件 → 认识表格 → 起草指标定义 → 计算 → 发现变化 → 分解原因
      → 检索文档 → 解释 → 给建议 → 【验证器复核】 → 生成报告
```

注意 `计算` 和 `验证器复核` 这两步是**完全不用 AI 的**，纯代码执行。AI 只在「理解」「解释」「组织语言」这几步出现。

2.3 三个必须先认识的名词

DuckDB：一个很轻的数据库，不需要装服务，直接对一个 CSV 文件执行 SQL 就能算数。你可以把它想成「一个特别会算表格的计算器」。

Fact（事实）：报告里出现的**每一个数字**都不是随手写的一句话，而是一条带身份证的记录。它长这样：

```
编号 N30
标签 2026-04 营收
数值 36200
怎么算的 SELECT SUM(...) FROM ... WHERE 月份='2026-04'
来源 指向源表的哪几行
是否已复核 true
```

也就是说，数字旁边永远挂着「我是怎么算出来的」。

验证器（verify）：报告发布前，逐条拿着 `怎么算的` 重新跑一次 SQL。如果存的是 36,200、重跑得到 35,000，就抛错，**整份报告不发布**。这叫 fail closed（出错就关门，绝不带着错数出门）。

2.4 一条特别关键的「规矩」

原项目源码里有一句非常重的注释，大意是：

> SQL 类事实，按存下来的参数重跑；**函数类事实，必须在一个叫 `RULES` 的登记表里有对应的「镜像重算规则」；查不到规则的函数没法被验证，这本身就是引擎的 bug。**

用大白话说：**你想加一个新算法，光把算法写好不算完，还得同时告诉查账员「这个结果该怎么独立重算一遍」；不告诉，系统直接拒绝出货。**

这条规矩是我们整个二次开发的落脚点，第 7 章会详细展开。

2.5 我们改了什么、没改什么

层	动了吗	说明
模型层 / 核心流水线 / 报告生成	**没动**	保持原样
验证器	只动 2 行 + 新增 1 个文件	加了 3 条镜像重算规则
数据层	新增	把无人机配送数据接进来
工具层	新增	3 个低空专属分析工具
评测层	新增	24 道题 + 4 轮实验

在代码仓库里用 `git status` 一眼就能看到这个边界——这对简历很重要，因为面试官最怕听到「我基于开源项目做了点东西」却说不清自己写了哪几行。

---

3 低空经济到底在讲什么

3.1 一个具体画面

深圳前海，一个物流园区（叫 depot）有一堆包裹要送。地面车太慢，于是用无人机从几个「起降点」（vertiport，可以理解成无人机的小机场）起飞，把包裹送到 23 个客户点，比如深圳大学、科技园、海岸城。

一天从早上 7 点运营到晚上 7 点，订单不是均匀来的：早高峰一阵、中午稀稀拉拉、傍晚又一阵（这叫「双峰」）。

3.2 为什么会有「禁飞区」

机场附近不能随便飞，大型活动场馆临时也管制。数据里有两个禁飞区：

禁飞区	形状	生效时间
深圳宝安机场（简化版）	5 个顶点连成的多边形	15:00—18:00
宝安体育馆附近临时管制	边长约 240 米的小方块	17:00—19:00

注意它们**只在特定时段生效**，这就是「时变禁飞区」。所以判断冲突不能只看「有没有飞进去」，还要看「飞进去的时候是不是正好在管制」。

3.3 这个项目想回答的三类问题

类型	举例	难点
纯数据聚合	「14:00—16:00 一共有多少单？」「哪个起降节点最忙？」	口径要一致（按什么分组、算不算重复订单）
空间 / 时空	「这条轨迹与禁飞区冲突几次？」「管制时段内有多少轨迹点？」	要判断「点在多边形内」，还要对上时间窗
复合推理	「需求最高的时段，和禁飞区生效时段重叠吗？」	要把前两类的结果再结合起来

---

4 数据从哪来，哪些是真、哪些是造的

这一章请认真读，因为**数据诚实度是这类项目最容易被面试官戳穿的地方**。

4.1 数据集叫什么

IEEE DataPort 上的一条记录，编号 DOI `10.21227/3gxb-mj63`，名字是「深圳前海 truck-drone 动态调度基准实例」，CC BY 4.0 授权（可公开使用，注明出处即可）。它描述的是：27 个服务节点、真实 WGS-84 经纬度、2 个时变禁飞区、雨季天气与道路拥堵、5 个运营日、每天约 52 单。

4.2 遇到的一堵墙

- **主数据压缩包（53.1 KB）需要 IEEE 账号登录**才能下载。
- 页面上给的配套代码仓库链接，实测是 **404（死链）**，跟网络、VPN 都无关。
- 所以那个「一键重建场景」的 Python 脚本因为缺依赖包，直接跑不起来。

4.3 绕过去的办法（这反而成了项目亮点）

页面上另外挂了一个**免登录的公开脚本包**，里面 24 KB 的 `build_case.py` 自带了全部「配方」：27 个真实坐标、禁飞区顶点与时段、订单到达的随机过程参数、3 个固定随机种子、拥堵表、天气转移矩阵。

我们做的事是：**用 AST（一种读代码结构的标准库）把这个脚本里的常量原样读出来**，再用 Python 标准库按脚本里写明的规则重新生成数据表。不复制粘贴硬编码、不执行来源不明的脚本、不需要任何账号。

一个很强的验证：用归档种子生成的**第 0 天恰好是 52 单**，跟 DataPort 摘要写的「每天 52 单」完全对上。其余 4 天换了种子，按随机过程自然波动成 39 / 55 / 48 / 39 单，合计 233 单。整个生成过程重跑两次，文件哈希一模一样。

4.4 三层数据，必须分清楚

层次	是什么	能不能说是「真实数据」
公开参数	27 个真实经纬度、禁飞区多边形与时段、订单生成规则、随机种子	✅ 来自公开发表
按规则生成	233 行订单（去哪、多重、什么时间窗）	✅ 可复现抽样，但不是现场记录
派生构造	2097 个轨迹点：从最近的起降点直线插值 9 个点、按哈希选巡航高度层、构造爬升/巡航/降落剖面	❌ **明确标注为我们造的合成轨迹，不是真实飞行记录**

第三层非常关键。原数据集给的是「节点」和「订单」，**没有真实飞行轨迹**。为了能测试「冲突检测」「高度剖面」这两个工具，我们必须自己造一批轨迹。我们在 `SOURCES.md`、报告、讲稿里反复写明它是 derived / synthetic，从不冒充实测数据。

> 简历上正确说法：「接入 IEEE DataPort 公开基准实例的真实坐标与禁飞区参数，并基于发布参数确定性派生配送轨迹」。千万别写成「使用真实无人机飞行数据」。

4.5 一个意外发现：禁飞区根本没挡住任何路线

我们原本准备测「冲突检测有多准」，结果算出来是 **0 次**。查下去发现不是程序写错，是数据本身如此：

```
机场禁飞区经度范围：113.792 ~ 113.842
全部 27 个节点、2097 个轨迹点的经度：≥ 113.845
```

也就是说，禁飞区在配送区域的**西边更远处**，两者在地图上根本没有交集。管制时段内确实有 1741 个轨迹点在飞，但没有一个落在区里。

这件事有两种做法：

- ❌ 偷偷改一下禁飞区坐标，制造几十次冲突，好让「提升」数字好看；
- ✅ 如实报告「该基准实例的禁飞区对当前配送网络无实际约束」，把它写成一条**数据质量结论**，另用**明确标注的合成探针轨迹**单独验证几何算法对不对。

我们选了第二种，还把它固化成一条回归测试。对数据分析岗来说，能发现并讲清数据的结构性局限，比刷一个漂亮数字值钱得多。

---

5 五张表长什么样

`lowalt/data/out/` 里有五张 CSV，可以用 Excel 直接打开。

文件	行数	主要列	用来干嘛
`nodes.csv`	27	节点号、中文名、类型（depot/vertiport/customer）、经纬度、本地米坐标	知道谁在哪
`no_fly_zones.csv`	9	区编号、顶点序号、经纬度、生效起止分钟与时刻	拼出两个多边形
`orders.csv`	233	订单号、第几天、释放时刻、时段桶、客户节点、重量、服务时长、时间窗	算需求
`trajectory_points.csv`	2097	轨迹号、序号、时间、经纬度、高度、高度层、阶段（地面/爬升/巡航/降落）、起降节点	算冲突与高度
`congestion.csv`	6	时段、速度系数	保留的公开参数

还有 `manifest.json`：记录每张表的行数、SHA-256、用了哪些公开参数、哪些是派生数据。**这个文件是数据可追溯性的凭证。**

时间怎么表示？统一用「从早上 7 点起算的分钟数」：`14:00 = 420`、`17:00 = 600`、`19:00 = 720`。所以题目里写「14:00—16:00」，代码里是 `release_min >= 420 AND release_min < 540`。注意右边是**开区间**（不含 16:00 整），避免边界订单被算两次。

---

6 三个新工具，一个一个拆

6.1 工具一：`drone_demand_aggregate`（需求聚合）

它回答：某个时间窗内有多少单？按小时怎么分布？哪个起降节点最忙？峰值在哪？

口径（写死在代码里）：

```
需求 = 时间窗内 COUNT(DISTINCT order_id)
起降节点 = 该订单轨迹 seq=0 那一点的 origin_node_id
分组 = (时段桶, 起降节点)
峰值 = 所有分组里订单数最大的那一组
```

一次真实运行（全部 5 天，14:00—16:00）：

```
窗口内 21 单
峰值 9 单，出现在 15:00-16:00，起降节点 1
分 5 个组、2 个时段、3 个起降节点
by_hour: 14:00-15:00 → 5 单；15:00-16:00 → 16 单
by_node: 节点 1 → 11 单；节点 3 → 8 单；节点 2 → 2 单
```

这个工具一口气发布了 **12 个 Fact**，每个数字（总量、每个时段、每个节点、峰值）都能被独立重算。

为什么要把口径写死？ 因为「配送需求」可以有很多种理解：算订单数还是算重量？按客户所在地分组还是按起飞点分组？含不含取消单？如果每次靠人临时写 SQL，同一个指标今天和明天可能不一样。写进工具后，口径只有一个。

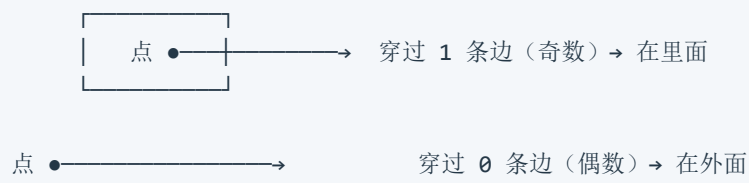
6.2 工具二：`no_fly_conflict_count`（禁飞区冲突）

它回答：某条轨迹与禁飞区冲突几次？命中多少点？累计多少分钟？管制时段内有多少轨迹点？

口径：

```
命中 = 轨迹点的经纬度落在禁飞区多边形内 AND 时间落在 [生效起，生效止)
一次冲突事件 = 同一条轨迹、同一个区里，一段连续命中点（序号连续）
输出 = 事件数、命中点数、累计分钟数、按区拆分、窗口内轨迹点数
```

「点在多边形内」怎么算？ 用一个经典办法叫**射线法（奇偶规则）**：从这个点向右画一条无限长的射线，数它穿过多边形几条边。穿过奇数次 → 在内部；偶数次 → 在外部。



我们特意用最基础的 Python 从头写了这个判断，没有调用任何几何库，理由是：这样才方便写**第二套完全独立的算法**去交叉验证（见 7.6）。

一个重要的口径选择：连续 9 个点都落在区里，算 **1 次冲突**，不是 9 次。因为业务上关心的是「这趟飞进行违规了几次」，不是「采样了多少个点」。这条定义写进了工具文档，也是基线容易答错的地方。

6.3 工具三：`altitude_distribution`（高度剖面）

它回答：巡航高度主要集中在哪一层？主导层占多少？巡航点占全部轨迹点多少？

口径：**只统计 `phase='cruise'` 的巡航点**，按 15 米一层（45/60/75/90 m）分组。

为什么要排除爬升和降落？因为轨迹里的点包含起飞（高度 0）、爬升（半层高）、降落，把它们混进来会污染「巡航高度分布」。这是一个**业务定义**，不是 SQL 能力问题——通用 SQL 也能写，但写的人得知道要加这个过滤。

第 0 天实测：主导层 `060-075m`，100 个巡航点，占 38.46%。

6.4 三个工具的共同点

每个数字都不是「一句话答案」，而是一条 `Fact`：带着计算 SQL 或函数名、参数、来源表。这正是下一章的基础。

---

7 验证器到底怎么保证数字可信

这一章是整个项目技术含量最高的部分，请慢一点读。

7.1 一条数字的旅程

```
工具算出 21
   ↓
生成 Fact：编号 N90 / 值 21 / SQL "SELECT COUNT(*) FROM orders WHERE ..."
           / 参数 [420, 540] / 来源 orders.csv
   ↓
写进报告前，验证器把这条 SQL 连同参数重新执行一遍
   ↓
重跑得到 21 → 一致 → 放行
重跑得到 19 → 不一致 → 抛错，整份报告不发布
```

这叫 **SQL 事实的复核**。简单、可靠、完全确定性。

7.2 但有些数字不是一条 SQL 能算出来的

比如「峰值 = 9」是从 5 个分组计数里取最大值；「主导层占比 = 38.46%」是 `最大组 / 总数 × 100`。这类数字叫**函数事实**，它的 `怎么算的` 不是 SQL，而是一个函数名 `fn` 加几个输入事实的编号。

回到 2.4 节那条规矩：**每个函数事实，都必须在 `RULES` 登记表里有对应的镜像重算规则**。我们新增了三条：

函数名	镜像规则（查账员怎么独立重算）
`lowalt.demand_peak.v1`	取所有分组计数里的最大值，**并额外检查报告里写的「峰值在 15:00-16:00、节点 1」确实对应那个最大值**
`lowalt.nfz_conflict.v1`	把物化表里按区的 SQL 计数加总
`lowalt.altitude_profile.v1`	`最大层计数 ÷ 各层总数 × 100`

如果某个新函数忘了登记，验证器会返回「查不到重算规则」并让整份报告失败——**这是框架的主动保护，不是我们额外加的。**

7.3 几何计算的麻烦：查账员只会 SQL

冲突检测里的「点在多边形内」是 Python 算的，不是 SQL。查账员不会射线法，怎么复核?

我们的办法是**把几何判断的结果物化成表**:

```
Python 逐点判断 → 写入 conflict_point_hits（每个命中点一行）
                → 按连续段合并，写入 conflict_events（每个冲突事件一行）
```

之后所有对外发布的数字，都是对这两张表做 `SELECT COUNT(*)` / `SUM(...)`。**于是只会 SQL 的查账员照样能重算**，「模型不碰算术」的哲学被保住了，可审计性也扩展到了几何计算。

7.4 但是——验证器不是万能的

这里有个非常容易被忽略、我们在报告和讲稿里反复强调的点:

> 验证器只保证「报告里的数 = 重新算出来的数」，**不保证「算的方法本身是对的」**。

如果射线法写错了，物化表也跟着错，SQL 重跑仍然一致，V2 照样 100% 通过。这就是所谓：**一致性 ≠ 正确性。**

7.5 所以补了第二层

手段	管什么
验证器 V2	一致性：数字有没有被抄错、计算有没有被篡改
独立第二实现	正确性：射线法本身有没有写错
Golden cases（标准答案用例）	边界情况：时间窗外、擦边、分段、未注册函数

独立第二实现用的是**角度和 / winding number**，跟射线法没有一行共享代码。在 41×41 = 1681 个网格点上，两套算法判定 **100% 一致**。

再加上 12 项单元测试：探针点分类、连续段分段、时间窗过滤、未注册函数是否 fail-closed、真实数据零交集回归、解析器主集回归、8 条改写题……全部通过。

7.6 合成探针：单独验证几何算法

因为真实数据冲突恒为 0（第 4.5 节），我们另造了 3 条**明确标注为 synthetic** 的探针轨迹，只用于算法单元测试，**从不混入业务评测题**：

探针	设计意图
A	在禁飞区外接矩形内、但在多边形外
B	真正穿过多边形内部，且时间在生效窗内
C	穿过多边形，但时间在生效窗外

结果：

方法	冲突事件数	命中点数
精确多边形（我们的工具）	**1**	**9**
外接矩形近似（基线）	2	18

外接矩形把「角上那块不属于禁飞区的空地」也算进去了，**高估 100%**。这就是通用工具链做空间分析时的典型失效。

---

8 评测怎么设计才不骗人

8.1 24 道题，分三层

类别	数量	例子
纯数据聚合	8	某窗口订单总量、峰值、按小时/节点分解、某天总量
空间 / 时空	8	冲突次数、命中点数、管制窗口内轨迹点数、主导高度层占比、巡航点数
复合推理	8	需求峰值时段是否与禁飞区生效时段重叠、晚高峰占全天比例、峰值分组占比

8.2 答案从哪来

每题的正确答案由**另一条独立编写的 SQL** 在原始表上算出，绝不读取工具的输出。这就像考试的标准答案由另一位老师单独做，不参考考生卷子。所有答案固化在 `golden.lock.json`，数据一变就能发现漂移。

8.3 最关键的一个决定：不把对手弄笨

很多项目会这样对比：「原版工具链不会算多边形，所以空间题全错 → 我们提升 100%」。这是**没有信息量**的，因为对手本来就没那个能力。

我们的做法反过来：

> **基线臂 = 一位会写任意 SQL 的分析师**，直接执行那条独立 SQL 答案键。它每道题都能答，数值上完全可以全对。

于是两版在「数值正确率」上打平是**预期之内**的，真正的差别只能来自别的地方：口径有没有固化、数字有没有被拆成可复核的事实、几何是不是精确、答案有没有带上维度和时间窗。

8.4 五个指标怎么算

指标	人话解释
数值正确率	答案和标准答案对不对得上
答案完整性	结构化打分：答案里有没有给「数值 / 属于哪个维度 / 时间窗 / 口径定义 / 分组明细」这几项，按满足比例计分。**不用另一个 AI 来打分**，避免引入新的随机性
可复核数字数	这次一共发布了多少个被 V2 逐条验过的 Fact
V2 通过率	验过的里面有多少一致
步数 / 耗时	工具调用几步、花了多少毫秒

为什么「答案完整性」值得单独算？因为只丢一个裸数字 `21` 和回答「14:00—16:00 共 21 单，峰值 9 单出现在 15:00-16:00 起降节点 1，按 5 个分组统计」，对业务方的信息量完全不同。

---

9 四轮实验，一轮一轮讲

9.1 第一轮：确定性主实验（不用 AI）

指标	基线	低空化版
数值正确率	**24/24**	**24/24**
答案完整性	0.399	**0.990**（+59.03 个百分点）
可复核数字	24	**242**（10.08 倍）
平均耗时	4.51 ms	25.20 ms

分层看，复合推理类的答案完整性从 0.333 涨到 0.969（**+63.55 个百分点**），可复核数字从 8 涨到 90。

这一轮最重要的其实是「没有提升的那一项」：数值正确率两版都是满分。我们完全可以设计一个算不对的基线来刷分，但没有。诚实地说：在确定性环境下，会写 SQL 的通用工具本来就能算对；二次开发的价值在**可审计性、口径一致性和答案完整性**。

9.2 第二轮：修好「听不懂人话」的问题

我们给评测加了一个自然语言解析器（把中文问题变成「调哪个工具、带什么参数」）。第一版按「关键词」判断，24 题只选对 19 个。

典型翻车：

> 问题：禁飞区活跃的 `15:00–18:00` 窗口内共有多少配送订单？
> 第一版：看到「禁飞区」→ 选了冲突检测工具 ❌
> 正确：这题问的是「多少订单」→ 应该选需求聚合 ✅

根因不是关键词不够，是**判错了对象**：应看「所求的量」，而不是「出现的话题名词」。第二版改成识别所求量，并显式返回三种状态：

状态	含义
`EXACT`	确定，直接执行
`AMBIGUOUS`	两个所求量得分接近，不硬猜，交给规划器
`UNRESOLVED`	没识别出问题在问什么，拒绝猜测

结果 **24/24 全 EXACT**。为了证明不是给那 5 道题打补丁，另写了 **8 条措辞完全不同的改写题**，8/8 通过；主实验三项指标一个没变，说明只动了解析层。单测从 8 项增到 12 项。

9.3 第三轮：把真正的 AI 接进来

前两轮完全没用大模型，好处是可复现、无噪声，坏处是回答不了「有 AI 的时候工具层还有没有用」。于是我们在本机跑了一个小模型。

环境：E 盘放 `llama.cpp` 推理引擎 + Qwen2.5-1.5B 权重，服务只监听 `127.0.0.1`，**不用任何 API key、数据不出本机**（Groq 打不开也没关系）。固定 `temperature=0`、`seed=42`，输出确定。

设计了四个臂，同一 24 题、同一答案键：

臂	做法	结果
① LLM 自己写 SQL	给它表结构和全部口径，让它写一条 SQL	**0/24**
② LLM 自己写 SQL（缩短提示）	只给本题相关口径，**对它是有利条件**	4/24
③ LLM 选工具（只做类型校验）	模型只选工具、填参数	7/24
④ **LLM 选工具（类型 + 问句约束校验）**	再加一层从问句出发的校验	**22/24**
参照：规则式工具层（无 AI）	第一轮	24/24

第 ① 臂为什么全错？逐条查过，不是解析 bug：模型把「14:00—16:00」当成 `day` 过滤、把订单量写成 `SUM(weight_kg)`、写出 `hour_bucket = 6` 这种类型错误。24 题里只有 7 条 SQL 能跑，其余 17 条直接执行失败。

第 ③ 臂为什么只有 7 个对？**模型几乎对每道题都填了 `day=4`**（把「全部 5 天」误读成「第 4 天」）。注意 `day=4` 是**合法值**（数据只有第 0–4 天），所以类型校验完全拦不住它。这叫**语义幻觉**：值合法，但意思是错的。

第 ④ 臂加的校验只读问题原文（到底有没有提到某一天、有没有时间窗、有没有具体禁飞区），不读答案、不读正确工具。加上之后 7 → 22。

剩下 2 题错在哪？C01、C03 选错了工具——**恰好就是第二轮里第一版解析器栽跟头的同两道题**。规则也好、模型也好，都会被「禁飞区活跃的窗口内有多少订单」这种复合措辞带偏；而第二版的「所求量」判定正是为它设计的。

9.4 第四轮：换个更大的模型，结论还成立吗

第三轮最容易被质疑的是「1.5B 太小了，换个强模型就没这事」。于是我们又下了 3B 和 7B，同题、同答案键、同提示词重跑：

模型	LLM 自己写 SQL	LLM 选工具	工具选择	每题耗时
1.5B	0/24 (0%)	**22/24**	22/24	4.3 秒
3B	2/24 (8.3%)	**23/24**	23/24	8.3 秒
7B	**5/24 (20.8%)**	**23/24**	23/24	17.1 秒
规则式工具层（无 AI）	—	**24/24**	24/24	0.005 秒

答案：阶梯**部分变平，但没有消失**。模型自己写 SQL 确实随规模变强（0 → 2 → 5），说明第三轮的 0/24 有一部分是模型能力问题，我们主动承认；但工具臂一直稳定在 22–23/24，差距从 22 题缩到 **18 题**，仍远未追平。三档里工具臂在聚合类和空间类都是满分，残差集中在复合推理类。

9.5 中途那个「7B 反而变差」的坑（很值得讲）

7B **第一次跑**工具臂只有 **10/24**，比 1.5B 的 22/24 还差，可工具选择是 23/24 几乎全对。这说明问题不在「选工具」，在后面的环节。

查出来原因很有意思：提示词里写了「不要编造时间窗」，**7B 太听话了**，干脆把参数整个留空 `{}`，工具于是退回「默认全天」，一堆题答成了全天的值。1.5B 反而乱填一气，被我们的校验纠正成了对的。

> **更遵守指令的模型，得分反而更低**——这不是模型变笨，是编排层只做了「删除非法参数」，没做「补全必要参数」。

修法：让参数处理同时干两件事——**剪枝**（丢掉幻觉的 day、区、时间窗）+ **补全**（从问题原文抽出确实提到的 day、时间窗、禁飞区填回去）。仍然不读答案、不读正确工具。

因为 `temperature=0` 输出是确定的，我们**用保存下来的原始模型输出离线重放**重新执行工具，没有重新推理、没有改动任何一个模型输出：

	重放前	重放后
1.5B	22/24	**22/24**（说明修正无副作用）
7B	10/24	**23/24**

修正前的原始结果都留着快照，可以随时复核。这条经验本身就是很好的面试素材：**编排层必须同时剪枝与补全，否则模型越听话越吃亏**。

---

10 一页看懂全部数字

项目	数字
数据	27 节点 / 2 禁飞区 / 5 天 233 单 / 2097 轨迹点；day0 精确复现 52 单
评测	24 题（8 聚合 + 8 空间 + 8 复合），答案由独立 SQL 给定
主实验	数值 24/24 → 24/24；答案完整性 0.399 → 0.990；可复核数字 24 → 242
复合推理类	答案完整性 0.333 → 0.969（+63.55 pp）；可复核数字 8 → 90
几何探针	精确 1 事件 / 9 点；外接矩形 2 事件 / 18 点（高估 100%）
独立校验	两套几何算法在 1681 点上 100% 一致；12 项单测全过
解析器	工具选择 19/24 → 24/24 全 EXACT；8 条改写题 8/8
本地 LLM 四臂	0 → 4 → 7 → 22/24（规则式 24/24）
模型规模	SQL 臂 0 → 2 → 5/24；工具臂 22 → 23 → 23/24
环境	本地 llama.cpp CPU，1.5B / 3B / 7B，**无 key，数据不出本机**

---

11 在你的电脑上把它跑起来

11.1 需要什么

项	要求
操作系统	Windows / Linux / macOS 都行（脚本是 Python）
Python	3.10 以上（开发时用 3.13）
Python 包	`duckdb`、`pyyaml` 两个，一条命令装好
浏览器	只在需要重新生成 PDF 时要 Chrome / Edge
跑本地大模型	可用内存 ≥ 4 GB；模型文件约 1–4 GB，需另下载

不需要：API key、账号、网络（除下载模型外）、Docker、GPU、管理员权限。

11.2 三条命令（确定性主实验，最快）

在交接包的 `project/` 目录里打开终端：

```
pip install duckdb pyyaml
python lowalt/data/build_dataset.py --out lowalt/data/out
python -m unittest discover -s lowalt/tests -v
python lowalt/eval/run_eval.py
```

预期看到：

```
Ran 12 tests ... OK
24 questions, 0 model calls
ALL numeric 24/24 -> 24/24 | relevance 0.399 -> 0.990 | verifiable numbers 24 -> 242
```

11.3 大模型怎么下、放哪

交接包**故意不包含大模型权重**（合计约 7.5 GB，不便传输）。详细地址和一键脚本见：

- `HANDOFF.md` 的「模型下载」一节
- `models/README.md`
- `tools/download_models.py`

最简流程：

```
python tools/download_models.py --all        # 下模型 + 推理引擎
python tools/check_env.py                    # 检查环境是否齐了
cd project
python lowalt/eval/run_llm_eval.py --start-server --model <模型路径> --tag 1p5b
```

如果新电脑没有 E 盘，不用改代码：设置环境变量 `LOWALT_MODEL_ROOT`（或分别设 `LOWALT_LLM_EXE`、`LOWALT_LLM_MODEL`）指向你放模型的目录即可。

11.4 常见问题

现象	原因 / 解决
`ModuleNotFoundError: duckdb`	没装依赖，`pip install duckdb pyyaml`
数据目录空 / 找不到 CSV	先跑 `build_dataset.py`；CSV 是生成物，不进 git
模型加载报 `CPU_REPACK buffer`	加 `--no-repack -c 1024`，脚本已默认带上
可用内存不足	关掉占内存大的程序；或换 0.5B / 1.5B 小模型
7B 跑得慢	正常，CPU 上约 17 秒/题，全量约 22 分钟
上游 TS 想编译	需 `bun install && bun run check`；当前机器 npm registry 不通，见 `HANDOFF.md` 未完事项

---

12 写在简历上 / 面试怎么讲

12.1 简历三行（可直接粘贴）

> 低空物流可审计分析 Agent 二次开发 | Python / DuckDB / SQL / 本地 LLM 评测
> - 基于开源可审计分析 Agent 二次开发，接入 IEEE DataPort 深圳前海 truck-drone 公开基准（CC BY 4.0），实现确定性数据接入（27 节点 / 233 单 / 2097 轨迹点，重跑哈希一致）。
> - 扩展需求聚合、禁飞区冲突检测、高度剖面 3 个工具并按上游契约注册 3 条验证器镜像重算规则，几何结果物化成 DuckDB 表供 SQL 复核。
> - 构建 24 题评测集与独立 SQL 答案键：答案完整性 0.399→0.990（复合类 +63.55 pp）、可复核数字 24→242；本地 1.5B/3B/7B 规模消融，模型自写 SQL 0→2→5/24、选工具稳定 22–23/24。

12.2 面试时最该主动说的三件事

1. **数值正确率没有提升，而且我们是故意让基线能写 SQL 的。**这比「提升 40%」更经得起追问。
2. **验证器只管一致性、不管正确性**，所以补了独立第二套几何算法和 golden cases。这是自己识别并补上的盲区。
3. **数据里禁飞区和配送网络根本没交集**，冲突恒为 0；没有硬凑数字，而是作为数据质量结论报告，另用合成探针验算法。

12.3 最可能被追问的问题

追问	要点
数据真实吗?	坐标/禁飞区/订单参数来自公开基准；轨迹是明确标注的派生构造，不是实测飞行
改了开源项目哪几行?	`recompute.ts` 改 2 处 + 新增 1 个规则文件，其余全在新目录 `lowalt/`
0/24 是不是故意让模型输?	两臂同样 schema 和口径；还给了 SQL 臂更有利的短提示消融；换 3B/7B 后确实涨到 2/5，主动讲
换强模型还成立吗?	部分成立：SQL 臂变强，工具臂稳定，差距仍 18 题；只覆盖本地 7B 以下小模型
上游跑通了吗?	Python 链路完整实测；Bun 依赖下载失败，TS 补丁未经编译验证，如实说明

更完整的 3 分钟讲稿和追问表在 `project/RESUME.md`。

---

13 诚实清单：局限与还没做完的事

局限	说明
模型小	1.5B–7B 本地 CPU 小模型；7B 用 Q3 量化（内存限制），5/24 是偏保守下界；不能推广到商用强模型
单次运行	`temperature=0 / seed=42`，每题一次，不报置信区间
工具臂带编排层	SQL 臂没有，所以不是纯模型能力对比——但这正是「工具层」的定义
数据	禁飞区零交集；轨迹为派生构造；天气事实表未生成
验证器边界	V2 只查一致性；几何正确性靠独立实现
性能	逐事实复核使聚合类耗时涨约 11 倍
未验证	上游 `bun install` 在当前网络失败，TS 补丁未经 `bun run check`
主归档	53.1 KB 原始数据需 IEEE 账号；本项目用免登录公开脚本参数复现

下一步优先级：① 换 7B Q4 / 更强模型复测；② 构造模糊提问测 `AMBIGUOUS` 命中率；③ 增量复核降 11× 耗时；④ 依赖可安装环境验证 TS 补丁；⑤ 接入有真实禁飞区交集的运营数据。

---

14 术语表

词	人话解释
Agent（智能体）	能理解任务、调用工具、组织答案的程序；这里特指分析 Agent
DuckDB	轻量级本地数据库，直接对 CSV 执行 SQL
SQL	用来向数据库提问的标准语言，如 `SELECT COUNT(*) FROM orders`
Fact（事实）	报告里每个数字的完整记录：值 + 怎么算的 + 来源
验证器 / V2	发布前把每个数字重算一遍的独立代码；对不上就不发
RULES	验证器里「函数类数字该怎么重算」的登记表
fail closed	出错就关门，绝不带错数出货
镜像重算规则	为新算法配一条独立的复核公式
WGS-84	全球通用的经纬度坐标系（GPS 用的就是它）
vertiport	无人机起降点，相当于小机场
NFZ / 禁飞区	No-Fly Zone，禁止或限制飞行的区域
射线法	判断点是否在多边形内的经典算法：向右画射线数穿边次数
外接矩形 / bbox	把不规则多边形简化成最小方框，会高估面积
物化表	把计算结果写成一张真实的表，方便后续用 SQL 复核
golden case	已知标准答案的测试用例，用来验算法对不对
baseline（基线）	对比实验里的「原版 / 对照组」
answer relevancy	本文用的结构化答案完整性评分，不依赖另一个 AI 打分
参数幻觉	模型编造出看起来合法、其实错误的参数（如 `day=4`）
剪枝 / 补全	编排层删掉非法参数 / 补上被省略的必要参数
GGUF	本地大模型权重的一种文件格式
llama.cpp	在普通电脑上跑大模型的轻量推理引擎
量化 Q4 / Q3	把模型权重压缩以省内存，数字越小越省但能力略降

词	人话解释
temperature=0	让模型输出确定、不随机
derived / synthetic	派生 / 合成数据，不是实测记录
CC BY 4.0	一种开放授权，可自由使用，需注明出处

---

15 文件地图：哪个文件是干什么的

路径	作用
`README_先看这个.md`	交接包总入口
`HANDOFF.md`	**详细交接文档**（环境、复现、模型下载、未完事项）
`01_论文_*.md / .pdf`	研究生论文形式的技术文章
`02_零基础教程_*.md / .pdf`	你正在读的这一篇
`project/`	**完整项目仓库**（可直接继续开发）
`project/EVAL_REPORT.md`	全部实验的量化报告（§1–§8）
`project/RESUME.md`	简历描述、3 分钟讲稿、追问表
`project/lowalt/README.md`	代码结构与四轮实验摘要
`project/lowalt/data/`	数据接入脚本、来源声明、生成的数据
`project/lowalt/engine/`	事实/验证器、三个工具、解析器、合成探针
`project/lowalt/eval/`	24 题、评测脚本、所有结果 JSON
`project/lowalt/tests/`	12 项单元测试
`project/lowalt/integration/`	上游 TS 补丁说明
`project/packages/verify/src/lowaltitude-rules.ts`	给上游验证器新增的 3 条镜像规则
`models/README.md`	**大模型下载地址与使用方法**
`tools/download_models.py`	一键下载模型与推理引擎
`tools/check_env.py`	环境自检
`tools/build_all.py`	一键复现全部确定性实验
`tools/md2pdf.py`	把 Markdown 转成本文档这样的 PDF

> 这份文档里的每个数字都能在 `project/lowalt/eval/results/` 里找到对应的 JSON。 如果你发现哪个数字对不上，那说明数据被改过或环境不同——请先跑一遍 `tools/check_env.py` 和 `tools/build_all.py`，再对照 `EVAL_REPORT.md`。
